СПЕЦКУРС

1 Книги

1. «Язык программирования C++» Страуструп
2. «Эффективное использование C++» Скотт Мэйерс
3. «Эффективный и современный C++»
4. «Чистый код» Роберт Мартин
5. «Чистая архитектура»

Почта: lusylvova@mail.ru

2 Лекция 1

Создатель C++ - Бьерн Страуструп

2.0.1 Язык C:

Требовался язык для работы с **аппаратными ресурсами**: Файлами, потоками…

Со временем, для реализации ООП добавили классы - 1979

- C++, он тогда назывался - C с классами

В 1984 C++ дали его нынешнее название

И дообавили:

1. Перегрузку
2. Проверку типов
3. public / private

В 1985 - 1й коммерческий выпуск C++

2.0.2 Релизы

98, 03, 11, 14, 17, 20, 23

Мы будем работать вплоть до 20го

Одна из целей разработки компилятора - обезопасить программиста до запуска программы

Ошибки на уровне компиляции.

1. C

   …
2. C с классами
   1. ООП
   2. UML диаграммы - проектирование программы
3. C++ с шаблонами

   Шаблон - это шаблон

   На основе шаблона, при указании типов, происходит кодогенерация компилятором целого класса / функции

   → Увеличивается объектник → Увеличивается время компиляции

Мы напишем собственную STL

Согласно cppreference - будут проверяться наши программы

2.0.3 STL (аббревиатура)

- Специализируемая библиотека шаблонов

- std::unique_ptr
- std::move
- std::vector
- std::map

2.0.4 Класс

- Это пользовательский тип, который определил программист

Встроенные типы:

- double
- float
- int
- char

Писать код следует в GNU style

```
class rational
  {
    // Интерфейс работы с классом
```

```
    public:
      rational (int num, int denom) // Конструктор
        {
          set (num, denom);
        }

      void set (int num, int denom) // сеттер
        {
           numerator = num;
           denominaator = denom;
        }
      ~rational () {} // = default;
      // Без явного задания - генерируется по-умолчанию
      float

    // Инвариант класса
    private:
      int64_t numerator = 0; // {}
      int64_t denomenator = {};
  }
```

```
class auto_file
  {
    FILE *fp;

    public:
      auto_file (const char *name, const char *mode)
        {
          fp = fopen (name, mode);
        }
      ~auto_file () { fclose (fp); }
      const FILE *get () const { return fp; }
  };

auto_file file ("file.txt", "r");
    if (!file.get());
      return 1;

    if (read_file (file.get() != RESULT_OK))
      return 2;

    if (func (...) != RESULT_OK)
      return 3;
```

Абстрактный класс -

```
class abstract
  {
    public:
      virtual func () = 0;
  };

// Производный (дочернний) класс
class some.class : public abstract
  {
    public:
      virtual func () override
      { ... }
  }
```

Инкапсуляция -
принцип, обеспечивающий скрытие деталий реализации

```
class Date
{
  public:
    Date (int dd, int mm, int yy)
    {
      set (dd, mm, yy);
    }

    void set (int dd, int mm, int yy)
    {
      day = dd;
      month = mm;
      year = yy;
    }

    void print () const;

  private:
    int day = 0; // {}
    int month {};
    int year {};
}
```

Решили поменять класс, цель оставить интерфейс тем же

```
class Date
{
  Date (int dd,  int mm, int yy)
  {
    //
  }

  void get (int &dd, int &mm, int &yy)
  {
  }

  private:
  int64_t epoch_sec {};
}
```

Другой пример:

```
class settings
{
  private:
  string m_hostname;
  int m_port {};

  public:
  settings (string hostname, int port)
  {
    m_hostname = hostname;
    m_port = port;
  }
  settings
}
```